

Day 27: Safe Amount Input

1. Day Number

Day 27

2. Topic Name

Basic Error Handling

3. Connection

You often convert amount input into a decimal number using `float()`.

For example:

```
amount = float(input("Enter amount: "))
```

This works when the user enters a valid number. However, the program crashes if the user enters text such as "hello".

Today, you will learn how to handle such wrong input safely using `try` and `except`.

4. Important Topics

- `try`
 - `except`
 - `float()`
 - Invalid input
 - `ValueError`
 - Friendly error messages
-

5. Foundational Notes

What is an error?

An error happens when Python cannot complete an instruction.

For example:

```
number = float("apple")
```

Python cannot convert "apple" into a number, so it raises a `ValueError`.

What does `try` do?

The `try` block contains code that might produce an error.

```
try:  
    # risky code
```

What does except do?

The except block runs when an error happens inside the try block.

```
except ValueError:  
    # handle invalid input
```

Why use ValueError?

float() raises a ValueError when the supplied text is not a valid number.

Examples:

```
"25"        → valid  
"25.50"     → valid  
"-10"       → valid number  
"hello"     → invalid  
""          → invalid
```

Important idea

Error handling prevents the program from suddenly crashing.

Instead of showing a technical error, your program can display a clear message such as:

```
Please enter a valid amount.
```

6. Easy Example

This example safely asks the user for their age:

```
try:  
    age = int(input("Enter your age: "))  
    print("Your age is:", age)  
except ValueError:  
    print("Please enter age using numbers only.")
```

What happens here?

1. Python asks for age.
2. int() tries to convert the input.
3. If the input is valid, the age is printed.
4. If the user enters text, the except block runs.
5. The program prints a friendly message instead of crashing.

7. Problem Statement

Create a Python program that:

1. Asks the user to enter an amount.
2. Uses try-except to convert the input into a float.
3. Prints the converted amount when the input is valid.
4. Prints a friendly error message when the input is invalid.

Do not repeatedly ask for input in this exercise. Handle only one input attempt.

8. Concepts Used

- Taking input with `input()`
 - Storing input in a variable
 - Converting text using `float()`
 - Handling errors using `try`
 - Catching `ValueError` using `except`
 - Printing a friendly result or error message
 - Defining and calling a function
-

9. Thought Process

Before writing the program, think through these steps:

1. Create a function for handling the amount input.
2. Ask the user to enter an amount.
3. Remember that `input()` always returns a string.
4. Place the `float()` conversion inside a `try` block.
5. If conversion succeeds, print the amount.
6. If conversion fails, catch the `ValueError`.
7. Print a simple and friendly error message.

The risky operation is:

```
float(user_input)
```

Therefore, that conversion should be protected by `try-except`.

10. Pseudocode

```
DEFINE a function for safe amount input

    ASK the user to enter an amount

    TRY
        CONVERT the input into a float
        PRINT the valid amount

    EXCEPT ValueError
        PRINT a friendly invalid-input message

CALL the function
```

11. Suggested Solving Approach

Use a **functional approach**.

Your program can have one main function responsible for:

- Asking for the amount
- Converting the amount
- Handling invalid input
- Printing the result

Suggested structure:

```
safe_amount_input()  
    get input  
    try conversion  
    handle ValueError  
    print result  
  
call safe_amount_input()
```

For this beginner exercise, the function does not need to return anything. It can directly print the result.

12. Easy Edge Cases

Case 1: User enters text

Input:

abc

`float("abc")` is invalid, so the `except` block should run.

Expected behaviour:

Please enter a valid amount.

Case 2: User enters nothing

Input:

An empty string cannot be converted into a float.

Expected behaviour:

Please enter a valid amount.

Case 3: User enters a decimal number

Input:

125.75

This is valid and should be converted successfully.

Expected behaviour:

```
Amount entered: $125.75
```

Case 4: User enters a whole number

Input:

```
500
```

`float()` can convert it into:

```
500.0
```

13. Expected Input

Valid input

```
Enter amount: 250.50
```

Invalid text input

```
Enter amount: twenty
```

Empty input

```
Enter amount:
```

14. Expected Output

For valid input

```
Enter amount: 250.50
Amount entered: $250.50
```

For invalid text

```
Enter amount: twenty
Please enter a valid amount.
```

For empty input

```
Enter amount:
Please enter a valid amount.
```

15. Hint Only

Place the conversion inside the try block:

```
amount = float(...)
```

Catch this specific error:

```
except ValueError:
```

Use a function name such as:

```
safe_amount_input()
```

Remember: only the code that might fail needs to be placed inside the `try` block.